

Бинарные деревья. Бинарные деревья поиска (цепочное и сплошное представление)

Миа Музамил Ахмад, 2 курс, группа 91
факультет ПММ,
ФГБОУ ВО «ВГУ»

Аннотация

- ❑ Цель работы:
 - Исследование BST и их представлений (цепочное и сплошное).
 - Реализация операций (вставка, удаление, поиск) на C++.
 - Сравнение эффективности представлений.
- ❑ Методы:
 - ❑ Рекурсия, стек, очередь.
 - ❑ Тестирование на различных входных данных.

Введение

- Ключевые тезисы:
 - Бинарные деревья — фундаментальная структура данных в информатике.
 - BST обеспечивают эффективные операции благодаря упорядоченности.
- Цель работы:
 - Исследовать два представления BST (цепочное и сплошное).
 - Реализовать базовые операции на C++.

Постановка задачи

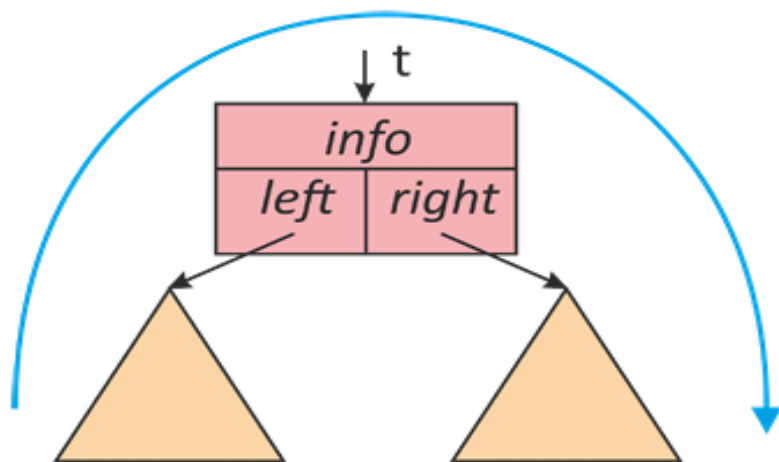
- ❑ Исследовать BST и их свойства.
- ❑ Реализовать:
 - Вставку, удаление, поиск.
 - Подсчёт чётных чисел и глубины дерева.
- ❑ Сравнить цепочное и сплошное представления.

Теоретическая часть

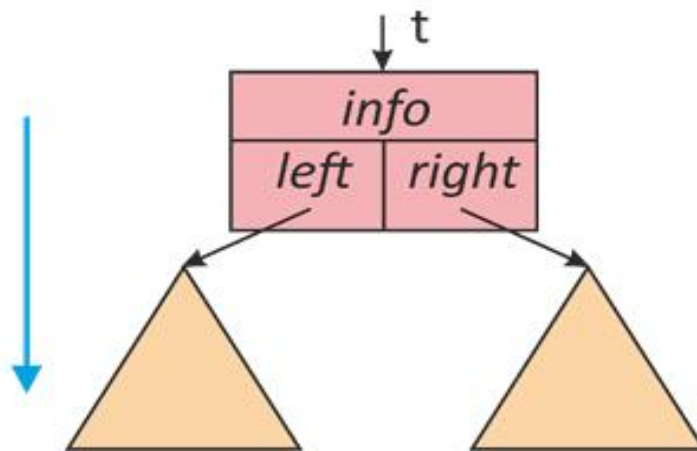
- ❑ Дерево — Это абстрактный тип данных для хранения информационных элементов имеющих нелинейное отношение.
- ❑ дерево — это связный ациклический граф, где:
 - Есть корневой узел.
 - Каждый узел (кроме корня) имеет ровно одного родителя.
 - Узлы могут иметь потомков.

Теоретическая часть

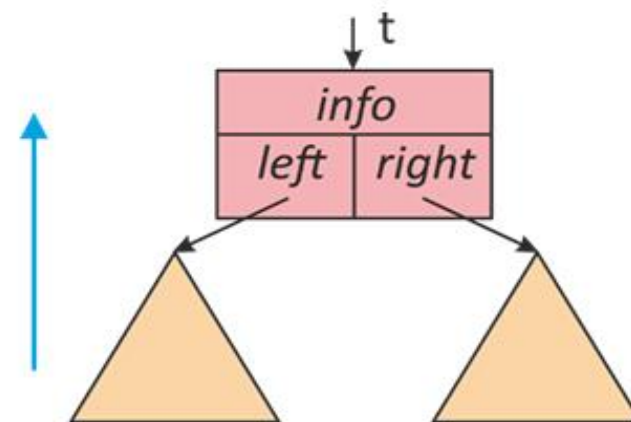
- обхода дерева:
 - Симметричный (In-order)
 - Прямой (Pre-order) или префиксный
 - Обратный (постфиксный) или Post-order



In-order обход



Pre-order



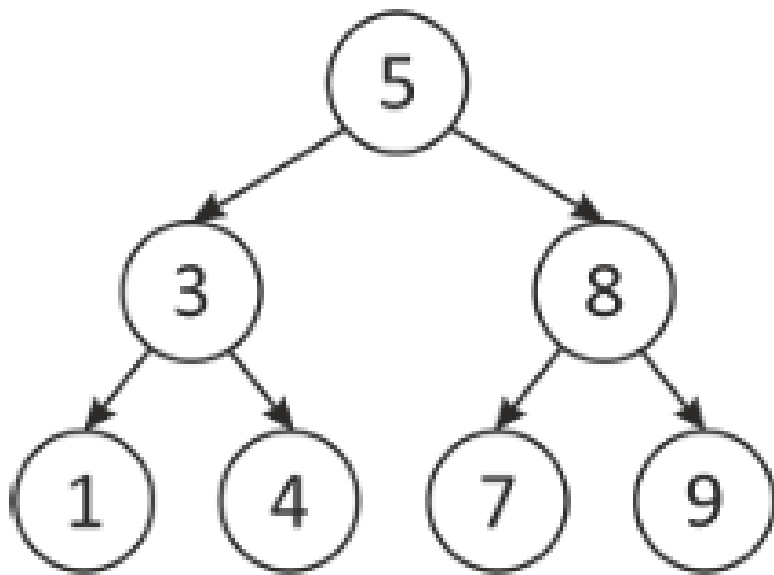
Post-order

Бинарные деревья поиска

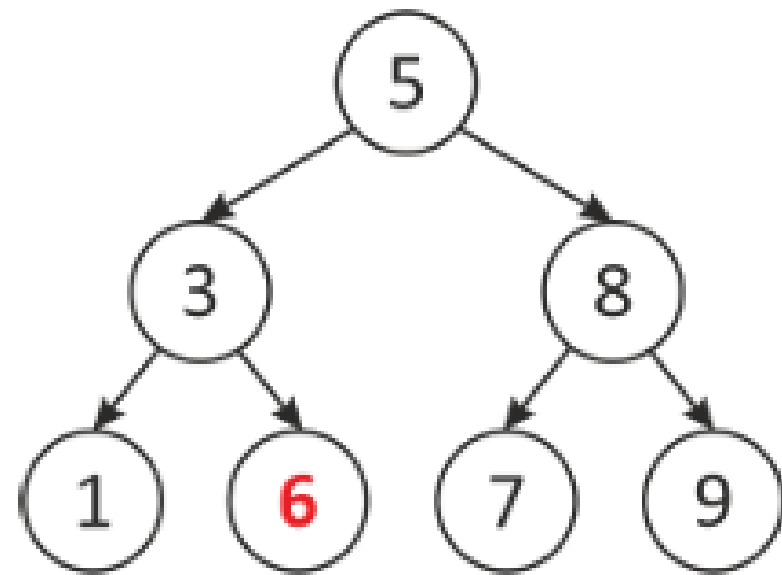
- *Дерево поиска (BST)* — это бинарное дерево, в котором для каждого узла выполняются следующие условия:
 - Ключи всех узлов в левом поддереве меньше ключа текущего узла.
 - Ключи всех узлов в правом поддереве больше ключа текущего узла.
 - Левое и правое поддеревья также являются бинарными деревьями поиска.

Бинарные деревья поиска

Дерево поиска



Не является деревом поиска



Основные операции

Вставка (Insertion)

- Алгоритм:
 - Если дерево пустое, создаём новый узел и делаем его корнем.
 - Иначе сравниваем ключ с текущим узлом:
 - Если ключ меньше, переходим в левое поддерево.
 - Если ключ больше, переходим в правое поддерево.
 - Повторяем, пока не найдём пустое место (NULL), куда вставляем новый узел.

Основные операции

Поиск (Search)

- Алгоритм:
 - Начинаем с корня.
 - Сравниваем искомый ключ с текущим узлом:
 - Если ключи равны, узел найден.
 - Если искомый ключ меньше, идём в левое поддерево.
 - Если искомый ключ больше, идём в правое поддерево.
 - Если доходим до NULL, ключ отсутствует.

Основные операции

Удаление (Deletion)

Удаление узла из BST сложнее, так как необходимо сохранить свойства дерева. Существует три случая:

1. Удаляемый узел — лист (нет потомков).
2. Удаляемый узел имеет одного потомка.
3. Удаляемый узел имеет двух потомков.

Основные операции

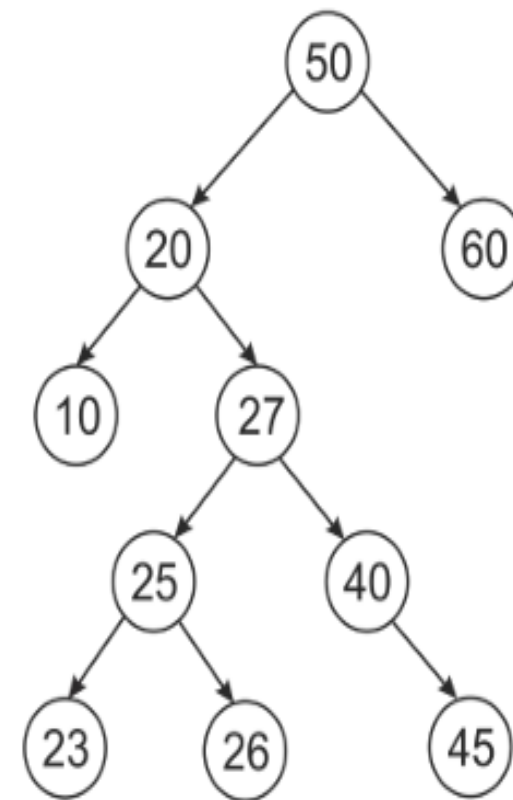
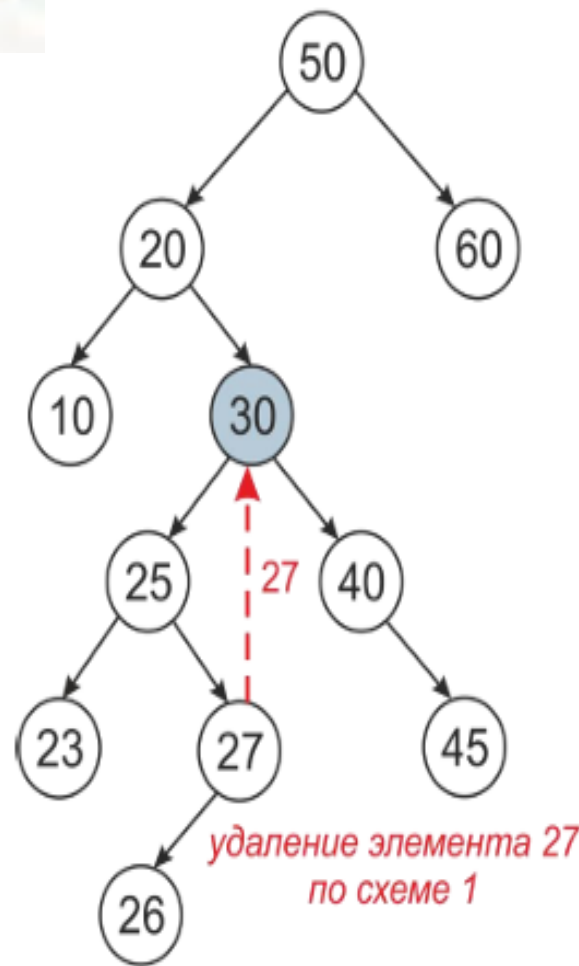
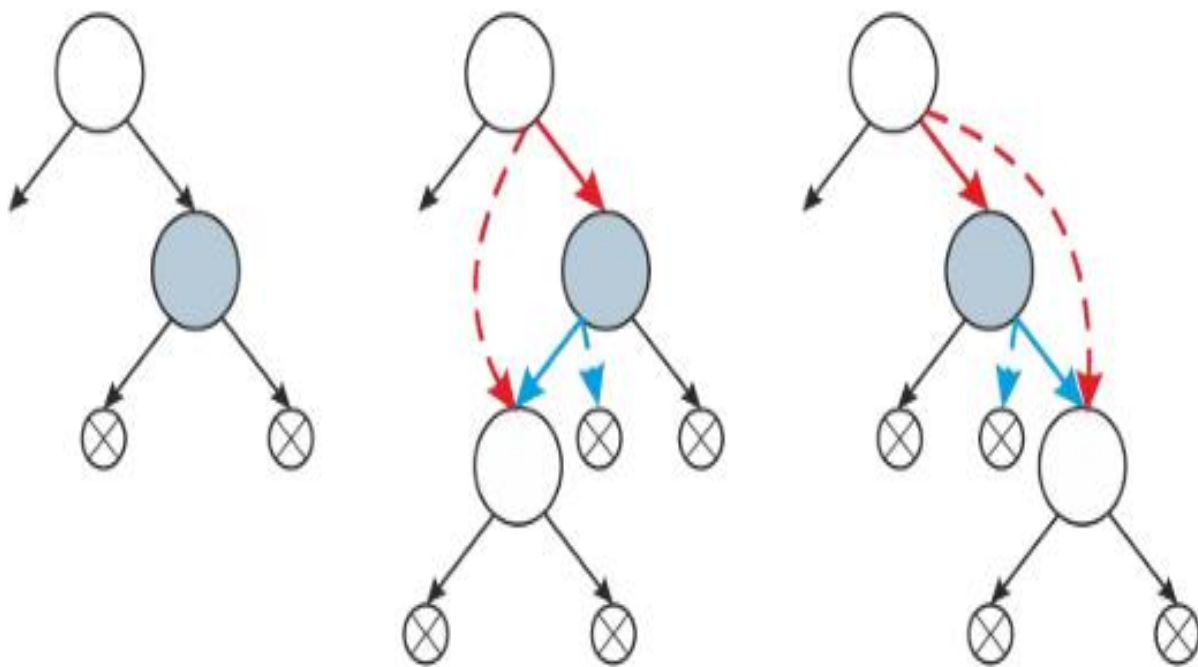
Удаление (Deletion)

□ Алгоритм:

- Найти узел для удаления (аналогично поиску).
- Обработать три случая:
 - **Лист:** Просто удаляем узел.
 - **Один потомок:** Заменяем узел его потомком.
 - **Два потомка:** Находим минимальный ключ в правом поддереве (или максимальный в левом), заменяем ключ удаляемого узла этим значением, затем удаляем найденный узел (он будет листом или иметь одного потока)

Основные операции

Удаление (Deletion)



Удаляемый элемент имеет двух потомков

Основные операции

Операция	Сложность (средний случай)	Сложность (худший случай)
Вставка	$O(\log n)$	$O(n)$
Удаление	$O(\log n)$	$O(n)$
Поиск	$O(\log n)$	$O(n)$

Цепочное представление

- *В цепочном представлении каждый узел содержит значение и указатели на левое и правое поддерево. Это динамическая структура, где память выделяется по мере добавления узлов. Структура узла обычно включает:*
 - *Ключ (значение узла).*
 - *Указатель на левое поддерево.*
 - *Указатель на правое поддерево.*

Сплошное представление

- В сплошном представлении бинарное дерево хранится в массиве, где связи между узлами определяются индексами:
 - Для узла с индексом i :
 - Левый потомок: $2i+1$.
 - Правый потомок: $2i+2$.
 - Родитель: $\lfloor (i-1)/2 \rfloor$. Этот подход наиболее эффективен для полных или почти полных деревьев, таких как бинарные кучи.

Сравнительный анализ

Характеристика	Цепочное представление	Сплошное представление
Хранение	Указатели на левое/правое поддерево	Массив с индексами
Память	Больше (указатели)	Меньше (без указателей)
Гибкость	Высокая (динамическое выделение)	Низкая (фиксированный размер)
Эффективность	Подходит для любых деревьев	Эффективно для полных деревьев
Сложность операций	$O(\log n)$ (сбалансированное)	$O(\log n)$ (для куч)
Фрагментация памяти	Возможна	Отсутствует

Insertion BST

```
#include<iostream>
#include<fstream>
#include<ostream>
#include<stack>
#include<queue>
using Tinfo = int;
struct NODE
{
    Tinfo info;
    int count = 1;
    NODE* left, * right;
    NODE(Tinfo info = 0, NODE* ptr_left = nullptr, NODE* ptr_right = nullptr)
        :info(info), left(ptr_left), right(ptr_right) {};
    ~NODE()
    {
        left = right = nullptr;
    }
};
```

```
void insert(Tree& t, Tinfo elem)
{
    if (!t)
        t = new NODE(elem);
    else
        if (elem < t->info)
            insert(t->left, elem);
        else
            if (elem > t->info)
                insert(t->right, elem);
            else
                ++t->count;
}

Tree Build_Search(std::ifstream& file)
{
    Tinfo elem;
    Tree root{};
    while (file >> elem)
        insert(root, elem);
    return root;
}
```

Search BST

```
//----- Search BST -----  
Tree Search(const Tree& root, Tinfo elem)  
{  
    Tree t = root;  
    Tree result = nullptr;  
  
    while (t != nullptr && result == nullptr)  
    {  
        if (elem < t->info) {  
            t = t->left;  
        }  
        else if (elem > t->info) {  
            t = t->right;  
        }  
        else {  
            result = t;  
        }  
    }  
  
    return result;  
}
```

Deletion BST

```
//----- Deletion BST -----  
void Find_elem(Tree& r, Tree& q)  
{  
    if (r->right)  
        Find_elem(r->right, q);  
    else  
    {  
        q->info = r->info;  
        q->count = r->count;  
        q = r;  
        r = r->left;  
    }  
}
```

```
bool Delete(Tree& t, Tinfo elem)  
{  
    bool result = false;  
    if (t) {  
        if (elem < t->info) {  
            result = Delete(t->left, elem);  
        }  
        else if (elem > t->info) {  
            result = Delete(t->right, elem);  
        }  
        else {  
            result = true;  
            if (t->count > 1) {  
                --t->count;  
            }  
            else {  
                Tree q = t;  
                if (!q->right) {  
                    t = q->left;  
                }  
                else if (!q->left) {  
                    t = q->right;  
                }  
                else {  
                    Find_elem(q->left, q);  
                }  
                delete q;  
            }  
        }  
    }  
    return result;  
}
```

Using Stack

```
// ----- Count Even Numbers Using the stack -----  
int count_even_Stack(Tree t){  
    std::stack<Tree> s;  
    int result{};  
    Tree root = t;  
    while (t){  
        result += (t->info % 2 == 0 ? 1 : 0);  
        if (t->left)  
        {  
            if (t->right)  
                s.push(t->right);  
            t = t->left;  
        }  
        else  
            if (t->right)  
                t = t->right;  
            else  
                if (s.empty())  
                    t = nullptr;  
                else  
                {  
                    t = s.top();  
                    s.pop();  
                }  
            }  
    }  
    return result;  
}
```

Using Queue

```
// ----- Count Even Numbers Using the Queue -----  
int count_even_Queue(Tree root)  
{  
    int result{};  
    std::queue<Tree> q;  
    Tree t{};  
    q.push(root);  
    while (!q.empty())  
    {  
        t = q.front(); q.pop();  
        result += (t->info % 2 == 0 ? 1 : 0);  
        if (t->left)  
            q.push(t->left);  
        if (t->right)  
            q.push(t->right);  
    }  
    return result;  
}
```



СПАСИБО!

ЕСТЬ ВОПРОСЫ?